

I want to watch your logs - how and why monitor .NET applications

Beata Zalewa

4Developers Kraków 2018

ABOUT

- In professional life consultant, Microsoft Certified Trainer, administrator, developer, freelancer, sometimes a miracle-worker and magician.
- From the beginning of career associated with Microsoft technologies.
- Private life in numbers: 1 husband, 1 daughter, 1 cat and 2 dogs. My hobbies are detective stories and photography.

Email: info@zalnet.pl

Web: <https://zalnet.pl/blog>

Linkedin: <https://linkedin.com/in/beatazalewa>

AGENDA

- Intro
- NLog
- .NET Memory Management
- BenchmarkDotNet package
- Working with LINQ
- MemoryDiagnoser
- Memory Leaks
- Roslyn-based heap allocation analyzer

MOTTO



Filipe Fortes
@fortes

Follow

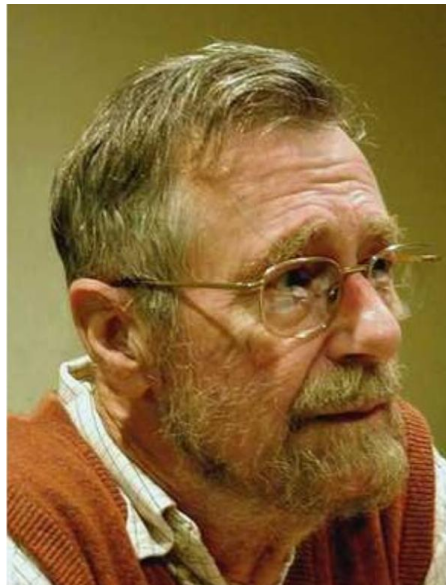


Debugging is like being the detective in a crime movie where you are also the murderer.

6:57 PM - 9 Nov 2013

SOMETIMES THINGS GO WRONG IN PRODUCTION

- Event Logs
- Performance Monitor
- Reproduce (Testing & Staging)
- **Capture errors and analyze**
- **Logging** is one of the most basic things that every application needs to have to help troubleshoot application problems.



If debugging is the process of removing software bugs, then programming must be the process of putting them in.

— Edsger Dijkstra —

AZ QUOTES

WHY SHOULD YOU USE C# LOGGING FRAMEWORK ?

- Most .NET developers know that they can write events to the Windows Event Viewer without implementing any extra libraries.
- This is the ultimate convenience and reliability.
- However, writing to the Event Viewer doesn't always work for some applications – for instance, you might want to write events to a log file to share with other departments or run analytics against a group of logs.
- Several logging frameworks promise .NET coders a “five-minute setup and start logging” solution for their projects.
- **Logging frameworks** are important because they make it easy to **send your logs to different places** by simply changing your configuration. You can write your logs to a text file on disk, a database, a log management system or potentially dozens of other places, all without changing your code.

LOGGING FRAMEWORK

- A logging framework is a utility specifically designed to standardize the process of logging in your application.
- This can come in the form of a third party tool, such as **log4j** or its .NET cousin, **log4net**.
- But a lot of organizations also roll their own.
- You have to go beyond just standardizing, however, to qualify as a framework.
- After all, you could simply “standardize” that logging meant invoking the file system API to write to a file called log.txt.
- A logging framework has to standardize the solution by taking it care of the logging for you, exposing a standard API.

POSSIBLE LOGGING FRAMEWORKS

- The list of .NET Logging Framework:
<https://github.com/quozd/awesome-dotnet/blob/master/README.md#logging>
Logging

- [Essential Diagnostics](#) - Extends the inbuilt features of System.Diagnostics namespace to provide flexible logging
- [NLog](#) - NLog - Advanced .NET and Silverlight logging
- [Logazmic](#) - Open source NLog viewer for Windows
- [ELMAH](#) - Official ELMAH site
- [Elmah MVC](#) - Elmah for MVC
- [Logary](#) - Logary is a high performance, multi-target logging, metric, tracing and health-check library for Mono and .NET. .NET's answer to DropWizard. Supports many targets, built for micro-services.
- [Log4Net](#) - The Apache log4net library is a tool to help the programmer output log statements to a variety of output targets
- [Sentry](#) - .NET SDK for [Sentry](#) Open-source error tracking that helps developers monitor and fix crashes in real time..
- [Serilog](#) - A no-nonsense logging library for the NoSQL era. Combines the best of traditional and structured diagnostic logging in an easy-to-use package.
- [StackExchange.Exceptional](#) - Error handler used for the Stack Exchange network
- [Semantic Logging Application Block \(SLAB\)](#) - Extends the inbuilt features of System.Diagnostics.Tracing namespace (EventSource class) to log to several sinks including Azure Tables, Databases, files (JSON, XML, text). Supports in-process and out-of-process logging through ETW, and Rx for real-time filtering/aggregating of events.
- [Foundatio](#) - A fluent logging API that can be used to log messages throughout your application.
- [Exceptionless](#) - Exceptionless .NET Client
- [Loupe](#) - Centralized .NET logging and monitoring. **[Proprietary]** **[Free Tier]**
- [elmah.io](#) - Cloud logging for .NET web applications using ELMAH. Find bugs before you go live. Powerful search, API, integration with Slack, GitHub, Visual Studio and more. **[Free for OSS]** **[\$]**
- [BugSnag](#) - Logs errors. Includes useful diagnostic info like stack trace, session, release, etc. Has a free tier. **[Free for OSS]** **[\$]**

TOP .NET LOGGING FRAMEWORKS ON THE MARKET

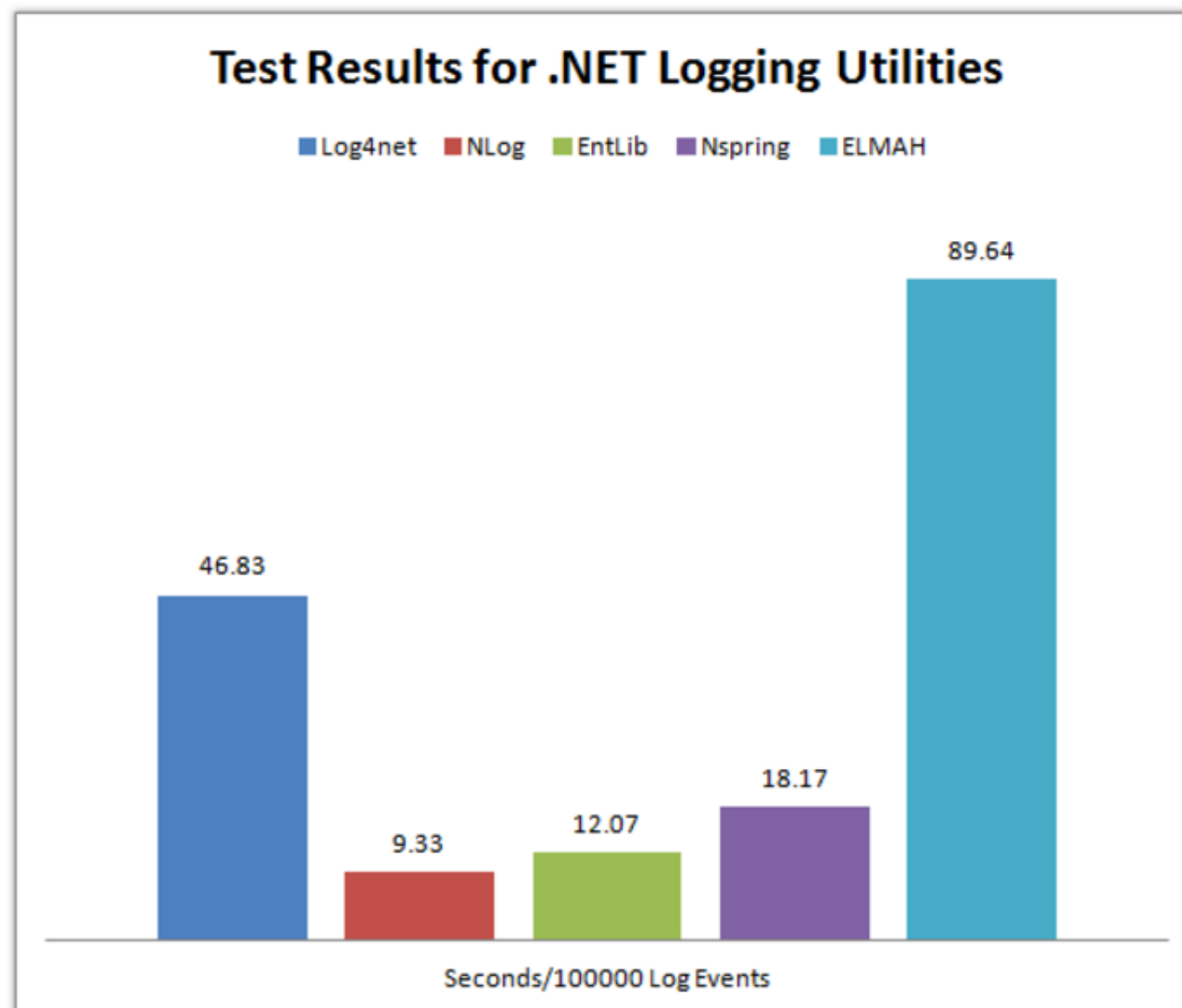
- NLog
- Log4net
- Microsoft Enterprise Library
- ELMAH
- NSpring
- Serilog
- log4net-loggly

TOP .NET LOGGING FRAMEWORKS ON THE MARKET

- Log4net, NLog, Microsoft Enterprise Library, and NSpring can be used in console applications.
- ELMAH is a web logger, so this application was run against a web application in .NET 4.5.
- C# is the primary language framework.
- All libraries are available in NuGet, so developers didn't need to download specific libraries and integrate DLLs and third-party solutions into the main project.

BENCHMARKS

(www.loggly.com)



BENCHMARKS

- **NLog** was the fastest. NLog truly did stand by its claim that you could start logging events in minutes. Getting NLog to work with the application took two lines of code: one line to instantiate the main logger class, and the other to log the events.
- **Log4net** was just as convenient, but it lost in terms of performance. It was just as convenient as NLog, but it couldn't compete on performance.
- **NSpring** was easy to set up, but it required much more code to work with it. You must open and close NSpring logger events, which you don't need to do with the others. A few more lines of code in a simple application aren't too much of a hassle, but opening and closing logger variables can be tedious for enterprise applications.
- The native **Microsoft Enterprise Library** has the second fastest times, but it was extremely inconvenient. Microsoft recently changed the way its framework works from version 5 to 6. In version 5.0, the developer could make a static call to log functions, but now there are numerous obstacles to get EntLib to work with version 6.0. The setup time for EntLib downgrades this library even though it was the second fastest.
- The final framework is **ELMAH**. ELMAH is a web logger, so it functions a bit differently than the others. It writes to an XML file by default, which could also explain some of its lag.

CONCLUSION AFTER BENCHMARKS

- For heavy-hitting applications that require file logging and speed, NLog was clearly the winner.
- NLog also has good support from the community with integrations for log management solutions like Loggly.
- If for any reason NLog isn't your choice, go with NSpring instead.
- These two frameworks are clearly the fastest.
- We would have chosen log4net as our top choice had it not been for the slow performance. It was one of the easiest to set up.
- All of these frameworks are available in NuGet, so every developer can try them out for himself.

FRAMEWORK₁

NLog

NLOG

- NLog is a very popular logging framework for .NET. It is second only to Log4net in popularity but is much newer and has a few unique features. In opposite to Log4net, NLog has a lot of great features and advantages.
- NLog is a free logging platform for .NET with rich log routing and management capabilities. It makes it easy to produce and manage high-quality logs for your application regardless of its size or complexity.
- It can process diagnostic messages emitted from any .NET language, augment them with contextual information, format them according to your preference and send them to one or more targets such as file or database.
- .NET, C/C++ and COM interop APIs are supported so that all your application components including legacy modules written in C++/COM can route their log messages through a common engine.
- NLog is open source software, licensed under the terms of BSD license.
- Project Homepage: nlog-project.org
- Project RSS Feed: nlog-project.org/feed

NLOG PACKAGES

NLog has several extension packages that add additional features.

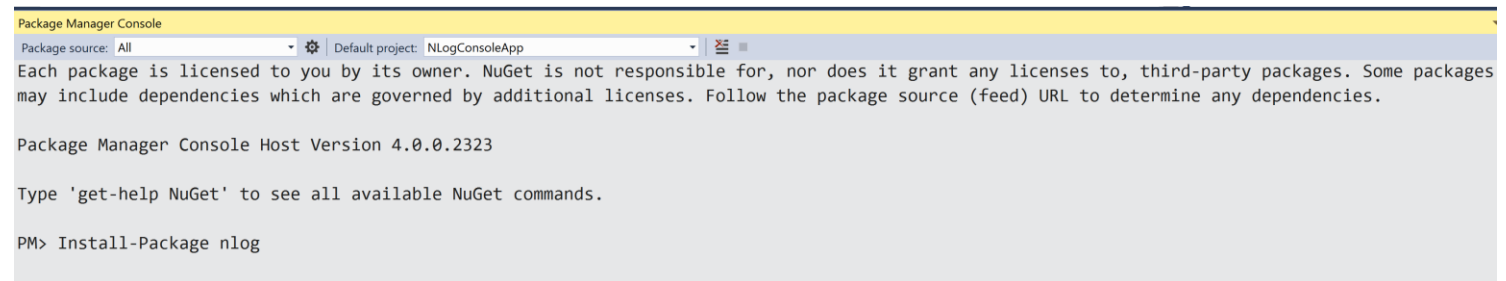
The following packages are available:

- **NLog.Web** – package contains targets and layout-renderers specific to ASP.NET 4 and IIS.
- **NLog.Web.AspNetCore** – package contains targets and layout-renderers specific to ASP.NET Core and IIS.
- **NLog.Extensions.Logging** – .NET Core support.
- **NLog.Windows.Forms** – package contains targets specific for Windows.Forms.
- **NLog.Extended** – MSMQ target, AppSetting layout renderer.
- **NLog.Config** – NLog Configuration example/starter – not supported in .NET Core.
- **NLog.Schema** – XSD Schema for the NLog config xml – for, among other, Intellisense in Visual Studio – not supported in .NET Core.

NLOG INSTALLATION

You can download and install NLog via NuGet

- Starting with NLog is as easy as installing an NLog NuGet package.
- You will also want to pick some logging targets to direct where your log messages should go, including the console and a text file.
- Just install **NLog.Config** package and this will install also **NLog** and **NLog.Schema** packages - this will result in a starter config and Intellisense.
- Use the GUI or the following command in the Package Manager Console:



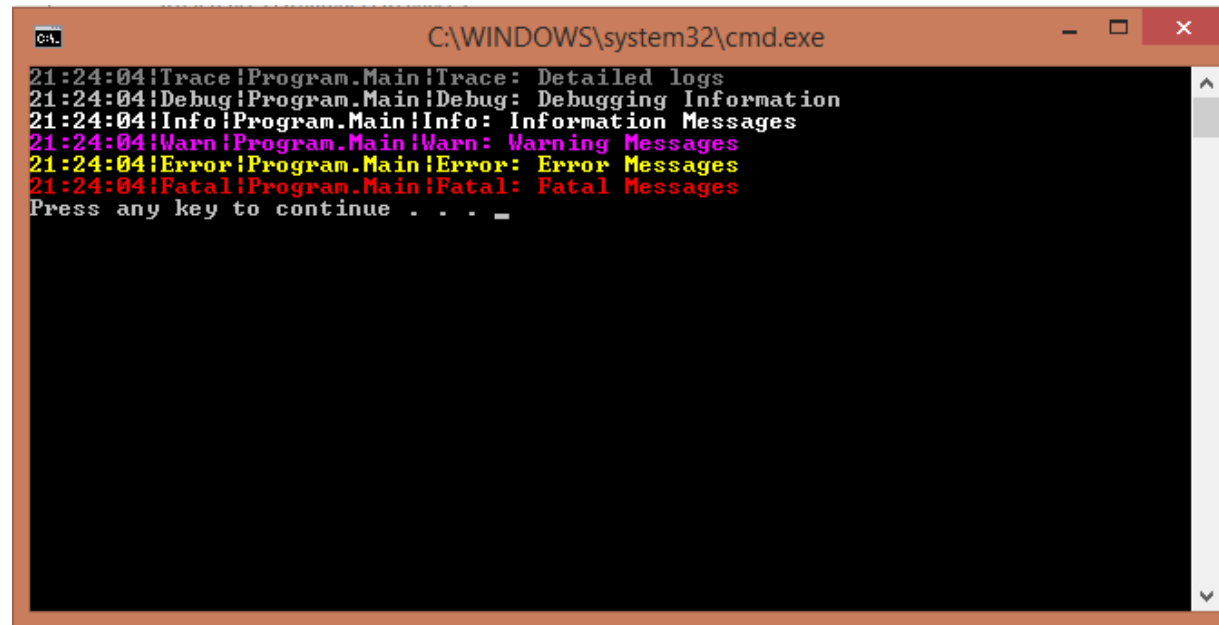
```
Package Manager Console
Package source: All | Default project: NLogConsoleApp
Each package is licensed to you by its owner. NuGet is not responsible for, nor does it grant any licenses to, third-party packages. Some packages may include dependencies which are governed by additional licenses. Follow the package source (feed) URL to determine any dependencies.
Package Manager Console Host Version 4.0.0.2323
Type 'get-help NuGet' to see all available NuGet commands.
PM> Install-Package nlog
```

- That's it, you can now compile and run your application and it will be able to use NLog

NLOG TARGETS

Logging Targets: What They Are and Common Targets You Need to Know

- Targets are how you direct where you want your logs sent.
- The most popular of the standard targets are most likely the File and Console targets.
- If you are using a Console, check out the ColoredConsole target:

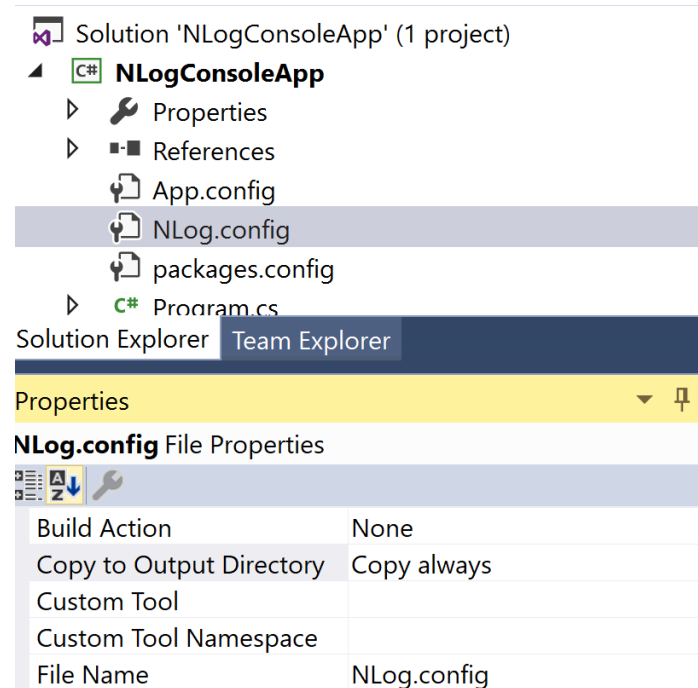


```
C:\WINDOWS\system32\cmd.exe
21:24:04!Trace!Program.Main!Trace: Detailed logs
21:24:04!Debug!Program.Main!Debug: Debugging Information
21:24:04!Info!Program.Main!Info: Information Messages
21:24:04!Warn!Program.Main!Warn: Warning Messages
21:24:04!Error!Program.Main!Error: Error Messages
21:24:04!Fatal!Program.Main!Fatal: Fatal Messages
Press any key to continue . . . _
```

NLOG TARGETS

Configure NLog Targets for output

- NLog will only produce output if having configured one (or more) NLog targets.
- NLog can be configured using XML by adding a NLog.config file to your application project (File Properties: **Copy Always**).



NLOG TARGETS

Available targets:

Config options for NLog's configuration [read more...](#)

search all 175 items

All platforms ▼

Targets 77

Layouts 4

Layout renderers 94

- **ColoredConsole** - Writes log messages to the console with customizable coloring.
- **Console** - Writes log messages to the console.
- **Debug** - Mock target - useful for testing.
- **File** - Writes log messages to one or more files.
- **Mail** - Sends log messages by email using SMTP protocol or pickup folder.
- **Mail** - Mail Target using MailKit. Only SMTP supported. **NLog.MailKit**
- **Null** - Discards log messages. Used mainly for debugging and benchmarking.

Async and buffering

- **AspNetBufferingWrapper** -  Buffers log events for the duration of ASP.NET request and sends them down to the wrapped target at the end of a request. **NLog.Web**
- **AsyncWrapper** -  Provides asynchronous, buffered execution of target writes.
- **AutoFlushWrapper** -  Causes a flush after each write on a wrapped target.

NLOG.CONFIG

This is a simple example of the content for NLog.config:

```
<?xml version="1.0" encoding="utf-8" ?>
| <nlog xmlns="http://www.nlog-project.org/schemas/NLog.xsd"
|   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
|
|   <targets>
|     <target name="logfile" xsi:type="File" fileName="file.txt" />
|     <target name="logconsole" xsi:type="Console" />
|   </targets>
|
|   <rules>
|     <logger name="*" minlevel="Info" writeTo="logconsole" />
|     <logger name="*" minlevel="Debug" writeTo="logfile" />
|   </rules>
| </nlog>
```

NLOG MESSAGES

Writing log messages

- Example of how to acquire a logger and writing a message to the logger:

```
var logger = NLog.LogManager.GetCurrentClassLogger();  
logger.Info("Hello World");
```

- If having configured the NLog targets correctly, then it will output the message to the configured targets.
- The logger has a name, which is used by the logging-rules (**name="*" is in configuration example above) to redirect to the wanted targets.**

NLOG MESSAGES

- When using **NLog.LogManager.GetCurrentClassLogger()**, then it will create a logger with the name of the calling class (with namespace). One can also specify an explicit name by using **NLog.LogManager.GetLogger("MyLogger")**.
- The logger can write messages with different LogLevels, which is used by the logging-rules (**minLevel** in configuration example above) so only relevant messages are redirected to the wanted targets.
- The logger is not tied to a specific target. The messages written to one logger can reach multiple targets based on the logging-rules configuration.
- Maintaining this separation lets you keep logging statements in your code and easily change how and where the logs are written, just by updating the configuration in one place.

NLOG LEVELS

Log levels

- Each log message has associated log level, which identifies how important/detailed the message is. NLog can route log messages based primarily on their logger name and log level.
- NLog supports the following log levels:

Trace - very detailed logs, which may include high-volume information such as protocol payloads. This log level is typically only enabled during development.

Debug - debugging information, less detailed than trace, typically not enabled in production environment.

Info - information messages, which are normally enabled in production environment.

Warn - warning messages, typically for non-critical issues, which can be recovered or which are temporary failures.

Error - error messages - most of the time these are Exceptions.

Fatal - very serious errors!

NLOG LAYOUT

Layouts and LayoutRenderers

It is possible to configure how log messages are written to a NLog target.

This shows the default SimpleLayout used by most NLog targets:

```
<target name="logfile" xsi:type="File" fileName="file.txt"  
layout="${longdate}|${level:uppercase=true}|${logger}|${messag  
e}" />
```

This can be configured to include more details:

```
<target name="logfile" xsi:type="File" fileName="file.txt"  
layout="${longdate}|${level:uppercase=true}|${logger}|${threadid  
}|${message}|${exception:format=tostring}" />
```

NLOG LAYOUT

Full list of Layout renderers is available on:

<https://nlog-project.org/config>



Configuration options

Config options for NLog's configuration [read more...](#)

search all 175 items

All platforms ▼

Targets 77

Layouts 4

Layout renderers 94

- `${cached}` -  Applies caching to another layout output.
- `${exception}` - Exception information provided through a call to one of the Logger methods
- `${level}` - The log level (e.g. ERROR, DEBUG) or level ordinal (number)

NLOG LAYOUT

One can also use a more complex Layout instead of SimpleLayout:

CsvLayout

JsonLayout

CompoundLayout

Log4JXmlLayout

Config options for NLog's configuration [read more...](#)

search all 175 items

All platforms ▼

Targets 77

Layouts 4

Layout renderers 94

- **CSV** - A specialized layout that renders CSV-formatted events.
- **Compound** - A layout containing one or more nested layouts.
- **JSON** - A specialized layout that renders to JSON.
- **Log4JXml** - A specialized layout that renders Log4j-compatible XML events.

NLOG WRAPPERS

- **Wrappers**

- NLog supports special kinds of targets which do not do any logging by themselves, but which modify the behaviour of other loggers. Those targets are called wrappers. The most commonly used ones are:
- **AsyncWrapper** - Improves Performance by providing asynchronous, buffered execution of target writes.
- **BufferingWrapper** - Simple batching of log messages. Maybe only send batch when certain log event occurs (Ex. Exception).
- **FallbackGroup** - Provides fallback-on-error.
- **RetryingWrapper** - Provides retry-on-error.

Async and buffering

- **AspNetBufferingWrapper** -  Buffers log events for the duration of ASP.NET request and sends them down to the wrapped target at the end of a request. 
- **AsyncWrapper** -  Provides asynchronous, buffered execution of target writes.
- **AutoFlushWrapper** -  Causes a flush after each write on a wrapped target.
- **BufferingWrapper** -  A target that buffers log events and sends them in batches to the wrapped target. Useful in combination with Mail target.

Databases

- **Database** - Writes log messages to the database using an ADO.NET provider.
- **LiteDB** - Writes NLog messages to LiteDB  
- **MongoDB** - Writes NLog messages to MongoDB  

Filtering, throttling and error handling

- **FallbackGroup** -  Provides fallback-on-error.
- **FilteringWrapper** -  Filters log entries based on a condition.
- **LimitingWrapper** -  Limits number of log events sent to target.
- **PostFilteringWrapper** -  Filters buffered log entries based on a set of conditions that are evaluated on a group of events.
- **RandomizeGroup** -  Sends log messages to a randomly selected target.
- **RepeatingWrapper** -  Repeats each log event the specified number of times.
- **RetryingWrapper** -  Retries in case of write error.
- **RoundRobinGroup** -  Distributes log events to targets in a round-robin fashion.
- **SplitGroup** -  Writes log events to all targets.

NLOG

Enable NLog's Own Internal Debug Logging

- If you are having any problems, you should enable the internal logging to help figure out the root cause.

```
<nlog internalLogFile="c:\log.txt" internalLogLevel="Trace">
  <targets>
    <!-- target configuration here -->
  </targets>
  <rules>
    <!-- log routing rules -->
  </rules>
</nlog>
```

NLOG RESULTS

- Results

C:\Windows\system32\cmd.exe

```
2018-10-14 21:41:16.9723|INFO|NLogConsoleApp.Program|Hello World
```

```
<?xml version="1.0" encoding="utf-8" ?>
| <nlog xmlns="http://www.nlog-project.org/schemas/NLog.xsd"
|   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
|
|   <targets>
|     <target name="logfile" xsi:type="File" fileName="file.txt" />
|     <target name="logconsole" xsi:type="Console" />
|   </targets>
|
|   <rules>
|     <logger name="*" minlevel="Info" writeTo="logconsole" />
|     <logger name="*" minlevel="Debug" writeTo="logfile" />
|   </rules>
| </nlog>
```

DEMO

NLog

FRAMEWORK 2

Log4net

LOG₄NET

What is Apache log₄net™

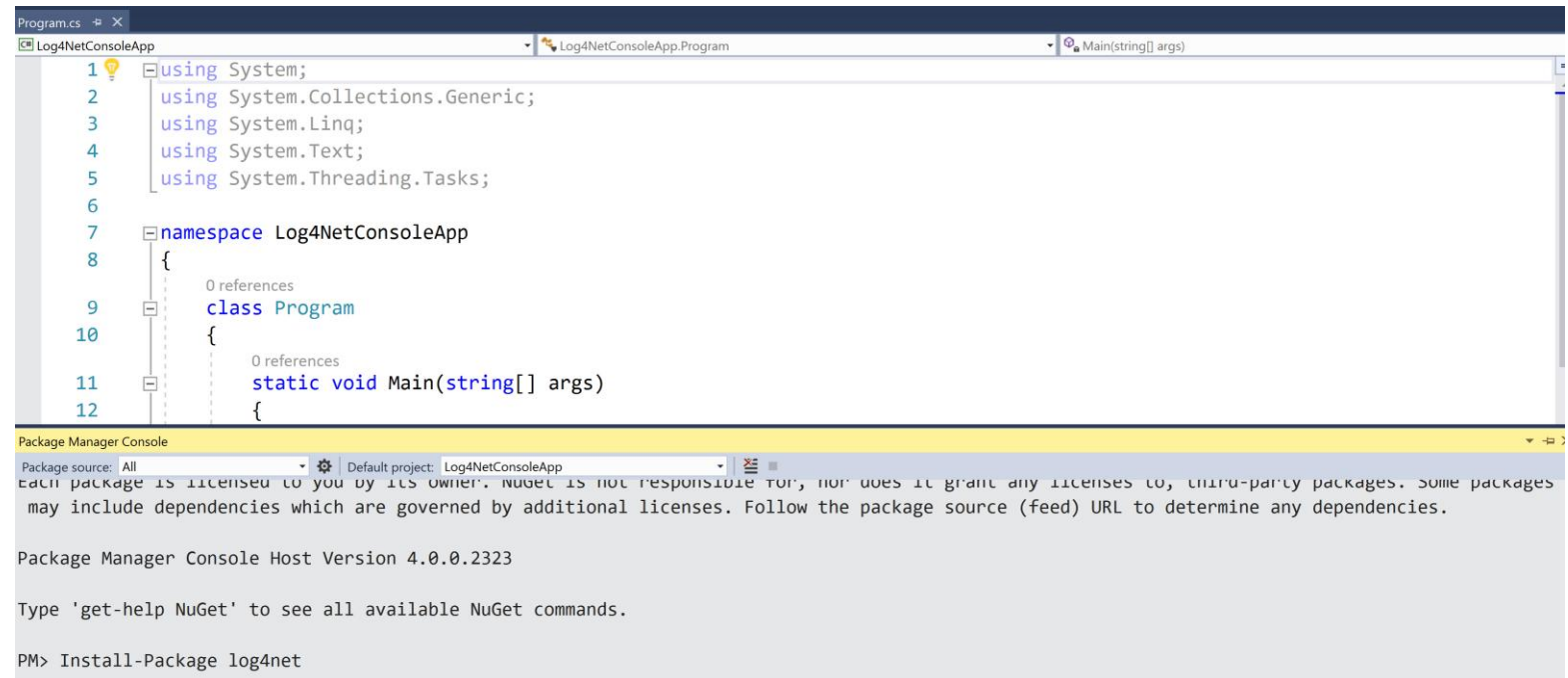
- The Apache log₄net library is a tool to help the programmer output log statements to a variety of output targets.
- log₄net is a port of the excellent Apache log₄j™ framework to the Microsoft® .NET runtime.
- The framework is similar in spirit to the original log₄j while taking advantage of new features in the .NET runtime.
- **The Apache log₄net project**
- log₄net is part of the **Apache Logging Services** project at the **Apache Software Foundation**.
- The Logging Services project is intended to provide cross-language logging services for purposes of application debugging and auditing.
- Project website: <http://logging.apache.org/log4net/>

HOW TO INSTALL LOG₄NET VIA NUGET

1. Add log₄net Package

- Starting with log₄net is as easy as installing a NuGet package. You can use the Visual Studio UI to search for it and install it, or just run this quick command from the Package Manager Console.

PM> Install-Package log4net



The screenshot shows the Visual Studio IDE. The top pane displays the code for `Program.cs` in the `Log4NetConsoleApp` project. The code includes several `using` statements for `System`, `System.Collections.Generic`, `System.Linq`, `System.Text`, and `System.Threading.Tasks`. It also defines a `namespace Log4NetConsoleApp` containing a `class Program` with a `static void Main(string[] args)` method. The bottom pane shows the Package Manager Console with the command `PM> Install-Package log4net` entered. The console output includes a warning about package licenses and the version of the Package Manager Console Host.

```
Program.cs X
Log4NetConsoleApp Log4NetConsoleApp.Program Main(string[] args)
1 using System;
2     using System.Collections.Generic;
3     using System.Linq;
4     using System.Text;
5     using System.Threading.Tasks;
6
7     namespace Log4NetConsoleApp
8     {
9         0 references
10        class Program
11        {
12            0 references
13            static void Main(string[] args)
14            {
15
16            }
17        }
18    }

Package Manager Console
Package source: All Default project: Log4NetConsoleApp
each package is licensed to you by its owner. NuGet is not responsible for, nor does it grant any licenses to, third-party packages. Some packages may include dependencies which are governed by additional licenses. Follow the package source (feed) URL to determine any dependencies.

Package Manager Console Host Version 4.0.0.2323

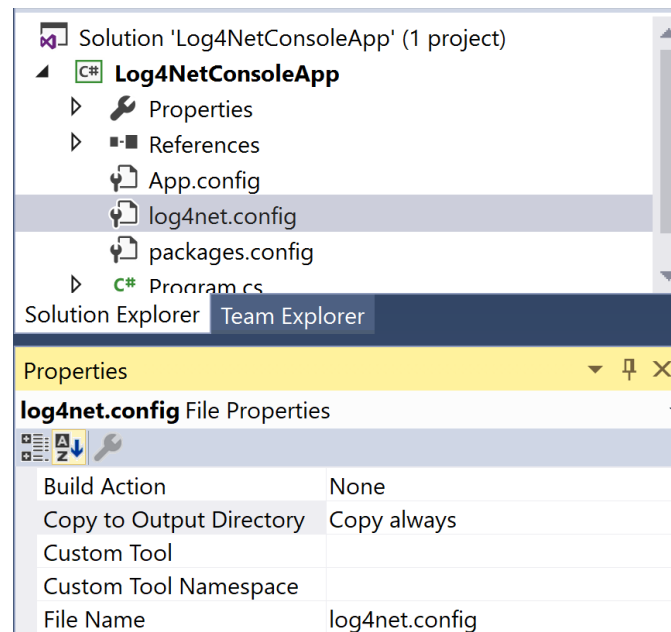
Type 'get-help NuGet' to see all available NuGet commands.

PM> Install-Package log4net
```

LOG₄NET. CONFIG

2. Add log₄net.config file

- Add a new file(*Application Configuration File*) to your project in Visual Studio called **log₄net.config** and be sure to set a property for the file. Set “Copy to Output Directory” to “Copy Always”. Then add content to this file.
- This is important because we need the **log₄net.config** file to be copied to the bin folder when you build and run your app.

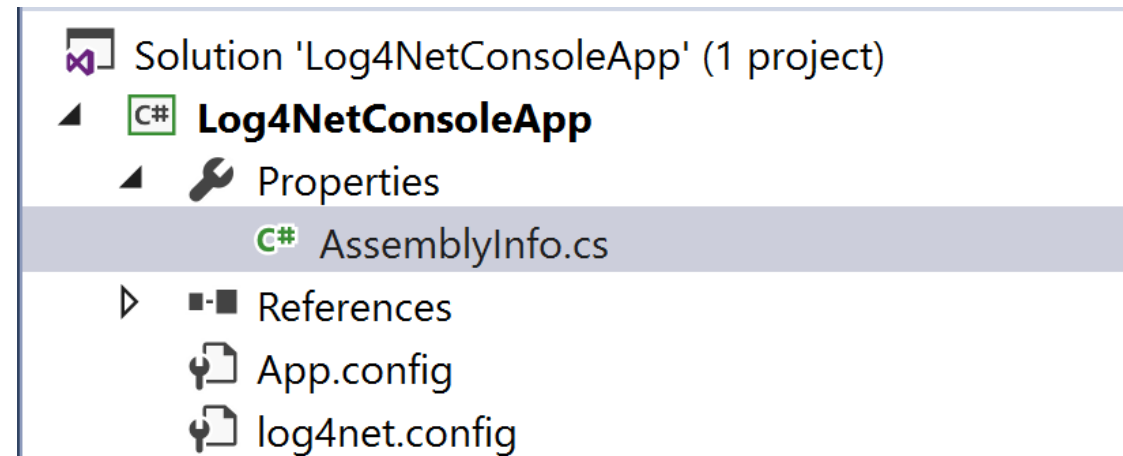


LOG₄NET

3. Tell log₄net to Load Your Config

- The next thing we need to do is tell log₄net where to load it's configuration from so that it actually works. I suggest putting this in your **AssemblyInfo.cs** file.
- Add this to the bottom of your AssemblyInfo file.

```
[assembly: log4net.Config.XmlConfigurator(ConfigFile = "log4net.config")]
```



LOG₄NET

4. Log Something!

- Now you can modify your app to log something and try it out!

```
using System;

namespace Log4NetConsoleApp
{
    0 references
    class Program
    {
        private static readonly log4net.ILog log = log4net.LogManager.GetLogger
            (System.Reflection.MethodBase.GetCurrentMethod().DeclaringType);

        0 references
        static void Main(string[] args)
        {
            log.Info("Hello logging world!");
            Console.WriteLine("Please, click the Enter key");
            Console.ReadLine();
        }
    }
}
```

LOG₄NET APPENDERS

5-

Log Appenders: What They Are and Which Ones You Need to Know

- Appenders are how you direct where you want your logs sent.
- The most popular of the standard appenders are most likely the `RollingFileAppender` and `ConsoleAppender`.
- I would also try the `DebugAppender` if you want to see your log statements in the Visual Studio Debug window so you don't have to open a log file.
- If you are using a Console, checkout the `ColoredConsoleAppender`.

<https://logging.apache.org/log4net/release/manual/introduction.html>

Type	Description
<code>log4net.Appender.AdoNetAppender</code>	Writes logging events to a database using either prepared statements or stored procedures.
<code>log4net.Appender.AnsiColorTerminalAppender</code>	Writes color highlighted logging events to a an ANSI terminal window.
<code>log4net.Appender.AspNetTraceAppender</code>	Writes logging events to the ASP trace context. These can then be rendered at the end of the ASP page or on the ASP trace page.
<code>log4net.Appender.BufferingForwardingAppender</code>	Buffers logging events before forwarding them to child appenders.
<code>log4net.Appender.ColoredConsoleAppender</code>	Writes logging events to the application's Console. The events may go to either the standard our stream or the standard error stream. The events may have configurable text and background colors defined for each level.
<code>log4net.Appender.ConsoleAppender</code>	Writes logging events to the application's Console. The events may go to either the standard our stream or the standard error stream.
<code>log4net.Appender.DebugAppender</code>	Writes logging events to the .NET system.
<code>log4net.Appender.EventLogAppender</code>	Writes logging events to the Windows Event Log.
<code>log4net.Appender.FileAppender</code>	Writes logging events to a file in the file system.

LOG₄NET LEVELS

5.

Log₄net levels:

- **All** – Log everything
- **Debug**
- **Info**
- **Warn**
- **Error**
- **Fatal**
- **Off** – Don't log anything

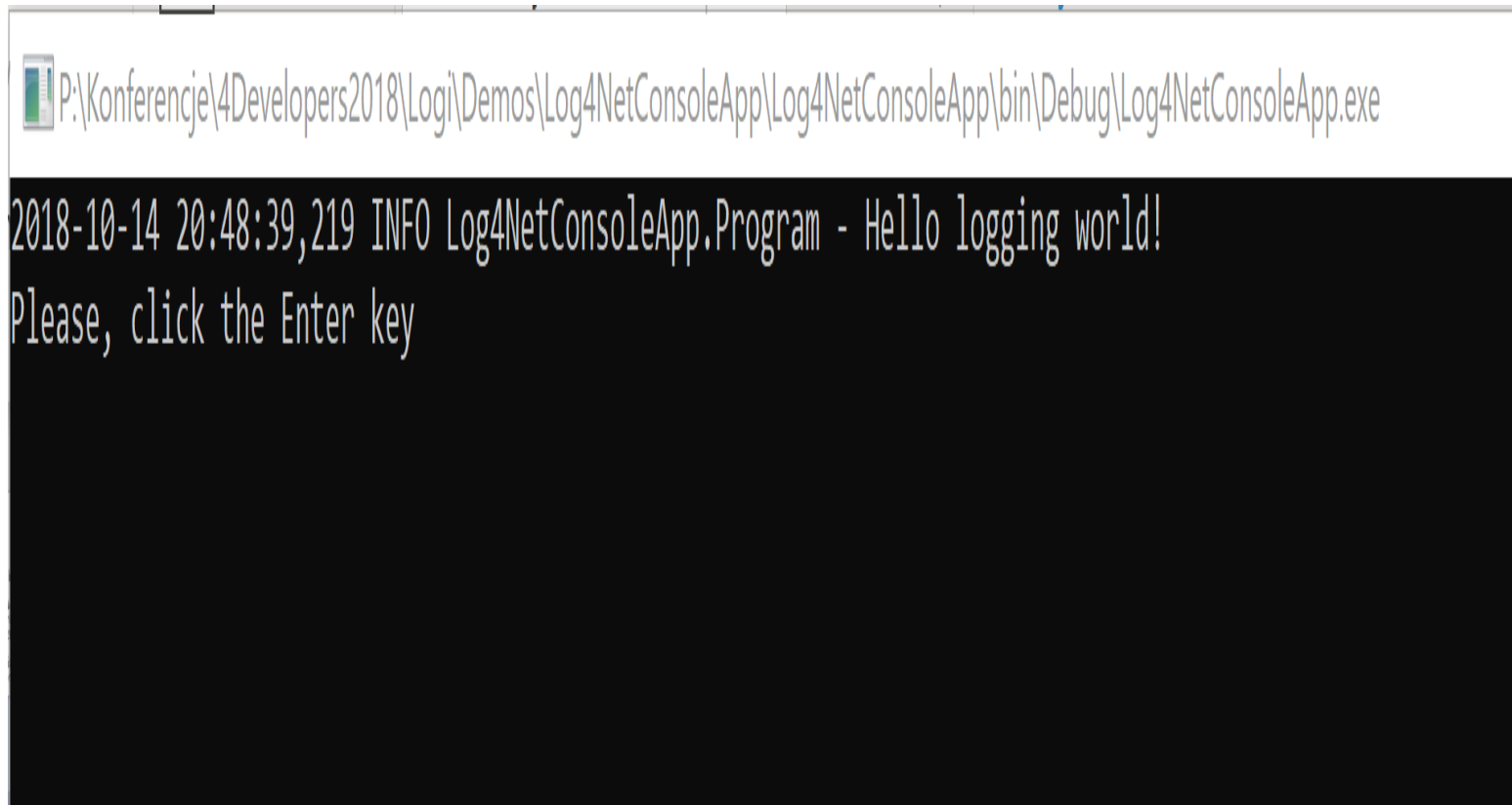
LOG₄NET LEVELS

Make Good Use of Multiple Log Levels and Filter by Them

- Be sure to use Debug, Info, Warning, Error, and Fatal logging levels as appropriate within your code.
- Don't log everything as Debug.
- Be sure to think about how you would be viewing the logs and what you want to see it later when coding your logging statements.
- You can specify in your log₄net config which log₄net logging levels you want to log.
- This is really valuable if you want to specify only certain levels to be logged to a specific log appender or to reduce logging in production.
- This also you to log more or less data without changing your code.

LOG₄NET

Result



```
P:\Konferencje\4Developers2018\Logi\Demos\Log4NetConsoleApp\Log4NetConsoleApp\bin\Debug\Log4NetConsoleApp.exe  
2018-10-14 20:48:39,219 INFO Log4NetConsoleApp.Program - Hello logging world!  
Please, click the Enter key
```

DEMO

Log₄net

FRAMEWORK₃

Serilog

SERILOG

- The newest logging framework, Serilog, was released in 2013.
- The big difference between Serilog and the other frameworks is that it is designed to do structured logging out of the box.
- Serilog is a good logging framework and it is built with the structured log data in mind.
- It is a kind of serializer. Serilog determines the right representation when the properties are specified in log events.
- Serilog provides variety of sinks such as file, MSSQL, Log4net, PostgreSQL, etc.

SERILOG

- **Structured logging example**
- Often you'll find that you're writing logs that contain two things: a message and value.
- **`log.Debug($"User id is: ${userId}");`**
- With most logging frameworks, this is simply translated to text in the log file. Text is nice and all, but knowing that the value logged was called `userId` and being able to search for that is actually very useful. Serilog keeps the properties available all the way to the destination.
- **`log.Debug("User id is {@userId}", userId);`**

SERILOG

- Serilog is backed by a commercial company <https://getseq.net/>.
- Retrace (log aggregator) also supports preserving the properties in the logs.
- Installing Serilog is slightly more complex than NLog or Log4net because it strives to be highly modular.

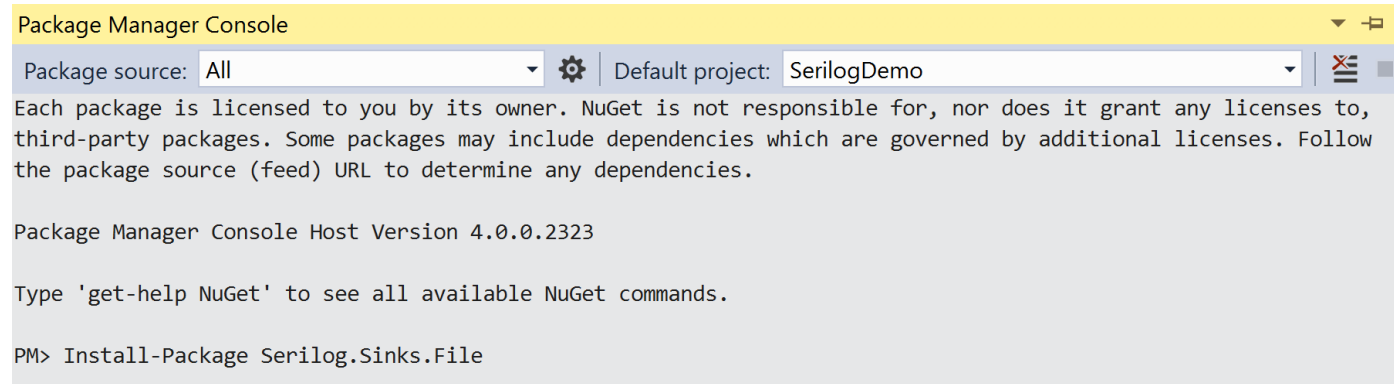
List of available sinks

* Denotes sink is maintained by the wider Serilog community

Sink Name	WriteTo.*	Package
Akka Actor *	AkkaActor	Serilog.Sinks.AkkaActor nuget v1.0.0.3 523
Alternate Rolling File *	RollingFileAlternate	Serilog.Sinks.RollingFileAlternate nuget v2.0.9 187.3k
Amazon CloudWatch *	AmazonCloudWatch	Serilog.Sinks.AwsCloudWatch nuget v4.0.138 230.5k
Amazon DynamoDB	DynamoDB	Serilog.Sinks.DynamoDB nuget v0.2.12 1.4k
Amazon Kinesis	AmazonKinesis	Serilog.Sinks.AmazonKinesis nuget v2.2.118 12.6k
Application Insights	ApplicationInsights	Serilog.Sinks.ApplicationInsights nuget v2.6.0 1.10m
Async Wrapper	Async	Serilog.Sinks.Async nuget v1.3.0 1.62m
Azure Analytics *	AzureAnalytics	Serilog.Sinks.AzureAnalytics nuget v3.1.0 54.9k
Azure DocumentDB	AzureDocumentDB	Serilog.Sinks.AzureDocumentDB nuget v4.0.0 66.5k

SERIOLOG LOGGING

- To write log data into the file, we need to use "Serilog.Sinks.File" dependency. It writes Serilog event to one or more file based on configuration. We can add this sinks by using either NuGet package manager or .net CLI.
- Using Package Manager

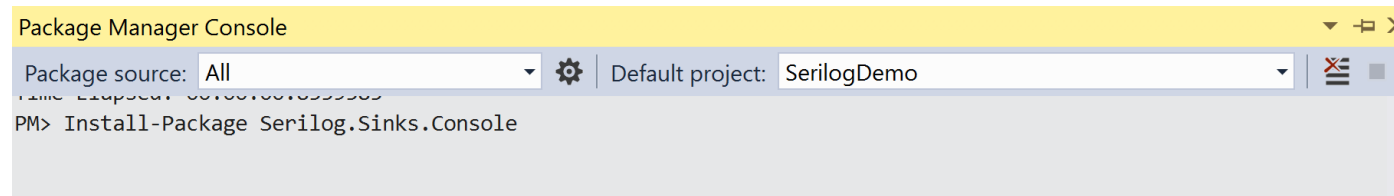


```
Package Manager Console
Package source: All [v] [g] Default project: SerilogDemo [v] [x] [y]
Each package is licensed to you by its owner. NuGet is not responsible for, nor does it grant any licenses to,
third-party packages. Some packages may include dependencies which are governed by additional licenses. Follow
the package source (feed) URL to determine any dependencies.

Package Manager Console Host Version 4.0.0.2323

Type 'get-help NuGet' to see all available NuGet commands.

PM> Install-Package Serilog.Sinks.File
```



```
Package Manager Console
Package source: All [v] [g] Default project: SerilogDemo [v] [x] [y]
Time Elapsed: 00:00:00.0000000
PM> Install-Package Serilog.Sinks.Console
```

SERILOG CONFIG

```
namespace SerilogDemo
{
    0 references
    class Program
    {
        0 references
        static void Main(string[] args)
        {
            Log.Logger = new LoggerConfiguration()
                .MinimumLevel.Debug()
                .WriteTo.Console()
                .WriteTo.File("logfile.log", rollingInterval: RollingInterval.Day)
                .CreateLogger();

            Log.Debug("Starting up");
            Log.Debug("Shutting down");

            Console.ReadLine();
        }
    }
}
```


SERIOLOG CONFIG

- By default, log file size limit is 1GB but it can be increased or removed using the `fileSizeLimitBytes` parameter of `WriteTo.File` method.

`.WriteTo.File("Logs/Example.txt", fileSizeLimitBytes: null)`

- We can also create log file per year, month, day, hour, or minute by specifying `rollingInterval` parameter to `WriteTo.File` method. In following example, I have defined `rollingInterval` to `RollingInterval.Day` so it will create new log file every day.

`.WriteTo.File("Logs/Example.txt", rollingInterval: RollingInterval.Day)`

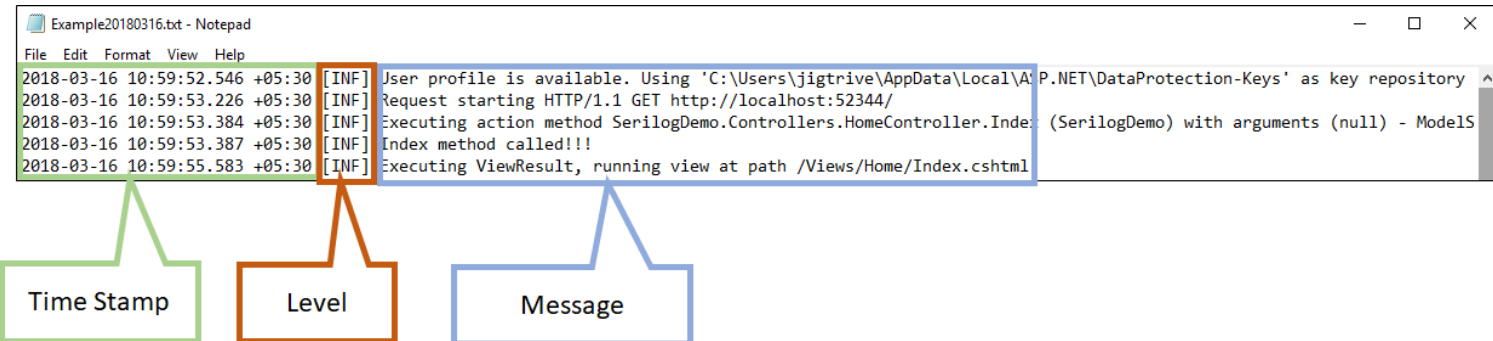
- Serilog retained most recent 31 files by default for some reason. We can change or remove this limit using `retainedFileCountLimit` parameter. In following example, I have assigned this parameter to `null`, so it removes limitation of 31 log files.

`.WriteTo.File("Logs/Example.txt", rollingInterval: RollingInterval.Day, retainedFileCountLimit: null)`

SERIOLOG CONFIG

Controlling Text file formatting

- The File sink creates events in sepecific fix format by default. Following is fixed format .



SERIOLOG LOGGING

- **Results:**

 P:\Konferencje\4Developers2018\Logi\Demos\SerilogDemo\SerilogDemo\bin\Debug\SerilogDemo.exe

```
[06:04:12 DBG] Starting up  
[06:04:12 DBG] Shutting down
```

DEMO

Serilog

FRAMEWORK 4

Elmah

ELMAH INTRO

- **What is ELMAH ?**

ELMAH stands for Error Logging Modules And Handlers that provide functionality to logging runtime ASP.NET errors.

- **Why do we choose ELMAH ?**

- ELMAH enables logging of all unhandled exceptions.
- ELMAH logs all errors in many storages, like - SQL Server, MySQL, Random Access Memory (RAM), SQL Lite, and Oracle.
- ELMAH has functionality to download all errors in CSV file.
- RSS feed for the last 15 errors
- Get all error data in JSON or XML format
- Get all errors to our mailbox
- Send error log notification to your application
- Customize the error log by customizing some code

ELMAH INTRO

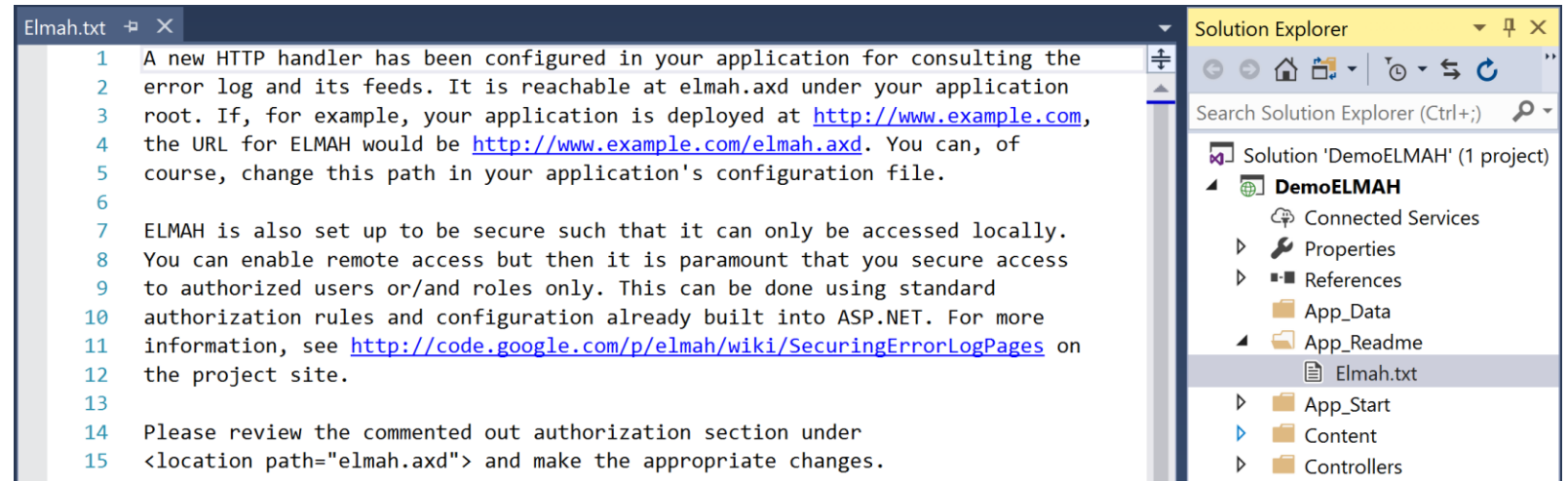
- **ELMAH HTTP Modules**

There are three HTTP Modules:

- *ErrorMailModule*
ErrorMailModule is used for sending the details of log as an email.
- *ErrorLogModule*
ErrorLogModule is used for logging all the exceptions and some other details, like IP- Address, Username, Website Username etc.
- *ErrorFilterModule*
ErrorFilterModule is used to customize the exception logs.

ELMAH INSTALL

- After successful installation, we will find the below screen.



The screenshot displays the Visual Studio IDE. The main editor window shows the 'Elmah.txt' file with the following content:

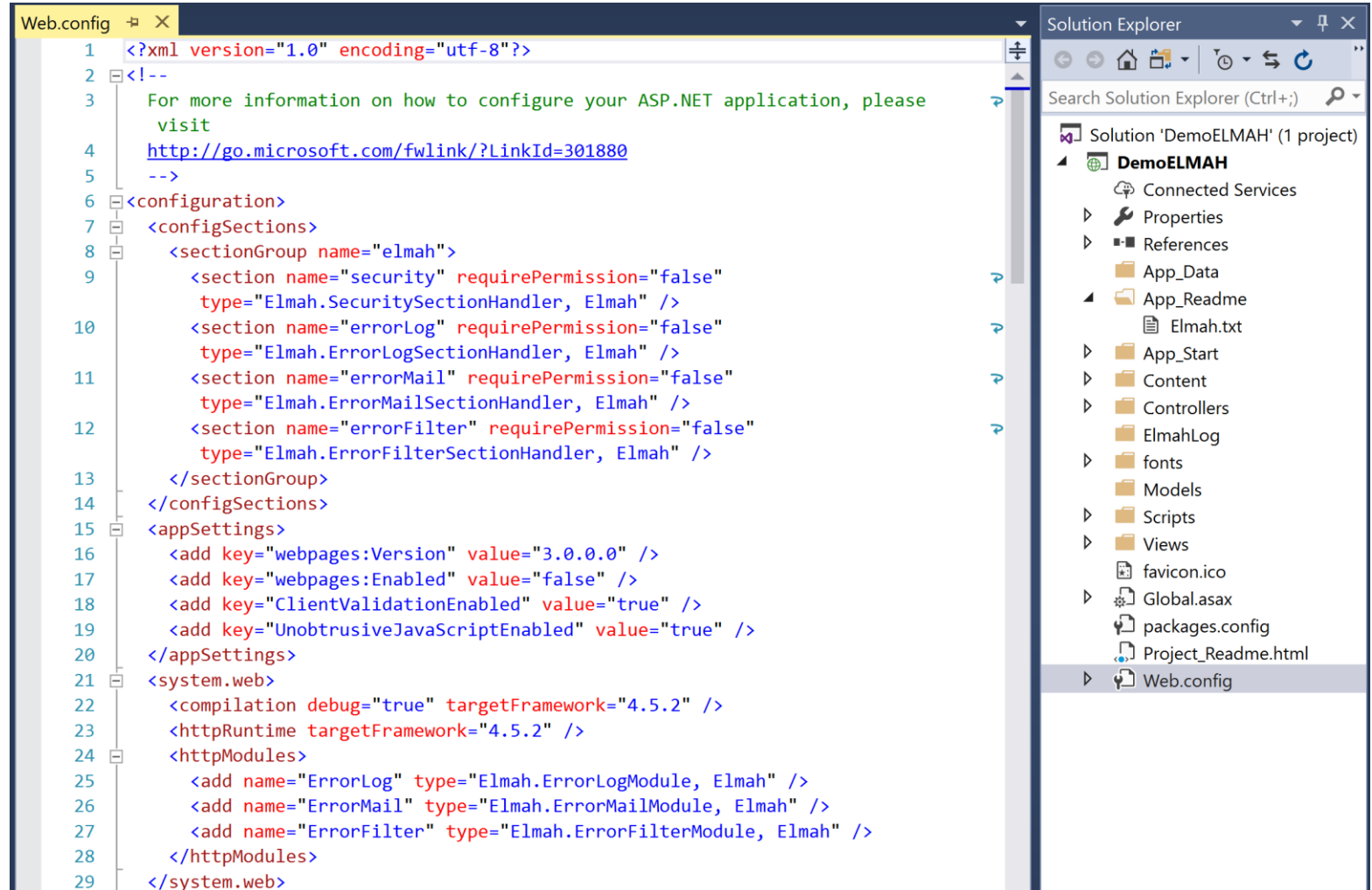
```
1 A new HTTP handler has been configured in your application for consulting the
2 error log and its feeds. It is reachable at elmah.axd under your application
3 root. If, for example, your application is deployed at http://www.example.com,
4 the URL for ELMAH would be http://www.example.com/elmah.axd. You can, of
5 course, change this path in your application's configuration file.
6
7 ELMAH is also set up to be secure such that it can only be accessed locally.
8 You can enable remote access but then it is paramount that you secure access
9 to authorized users or/and roles only. This can be done using standard
10 authorization rules and configuration already built into ASP.NET. For more
11 information, see http://code.google.com/p/elmah/wiki/SecuringErrorLogPages on
12 the project site.
13
14 Please review the commented out authorization section under
15 <location path="elmah.axd"> and make the appropriate changes.
```

The Solution Explorer on the right shows the project structure for 'Solution 'DemoELMAH' (1 project)'. The project 'DemoELMAH' contains the following items:

- Connected Services
- Properties
- References
- App_Data
- App_Readme
 - Elmah.txt
- App_Start
- Content
- Controllers

ELMAH CONFIG

- ELMAH Modules are default registered in web.config.



```
Web.config
1 <?xml version="1.0" encoding="utf-8"?>
2 <!--
3   For more information on how to configure your ASP.NET application, please
4   visit
5   http://go.microsoft.com/fwlink/?LinkId=301880
6 -->
7 <configuration>
8   <configSections>
9     <sectionGroup name="elmah">
10       <section name="security" requirePermission="false"
11         type="Elmah.SecuritySectionHandler, Elmah" />
12       <section name="errorLog" requirePermission="false"
13         type="Elmah.ErrorLogSectionHandler, Elmah" />
14       <section name="errorMail" requirePermission="false"
15         type="Elmah.ErrorMailSectionHandler, Elmah" />
16       <section name="errorFilter" requirePermission="false"
17         type="Elmah.ErrorFilterSectionHandler, Elmah" />
18     </sectionGroup>
19   </configSections>
20   <appSettings>
21     <add key="webpages:Version" value="3.0.0.0" />
22     <add key="webpages:Enabled" value="false" />
23     <add key="ClientValidationEnabled" value="true" />
24     <add key="UnobtrusiveJavaScriptEnabled" value="true" />
25   </appSettings>
26   <system.web>
27     <compilation debug="true" targetFramework="4.5.2" />
28     <httpRuntime targetFramework="4.5.2" />
29     <httpModules>
30       <add name="ErrorLog" type="Elmah.ErrorLogModule, Elmah" />
31       <add name="ErrorMail" type="Elmah.ErrorMailModule, Elmah" />
32       <add name="ErrorFilter" type="Elmah.ErrorFilterModule, Elmah" />
33     </httpModules>
34   </system.web>
```

Solution Explorer

Solution 'DemoELMAH' (1 project)

- Connected Services
- Properties
- References
- App_Data
- App_Readme
 - Elmah.txt
- App_Start
- Content
- Controllers
- ElmahLog
- fonts
- Models
- Scripts
- Views
- favicon.ico
- Global.asax
- packages.config
- Project_Readme.html
- Web.config

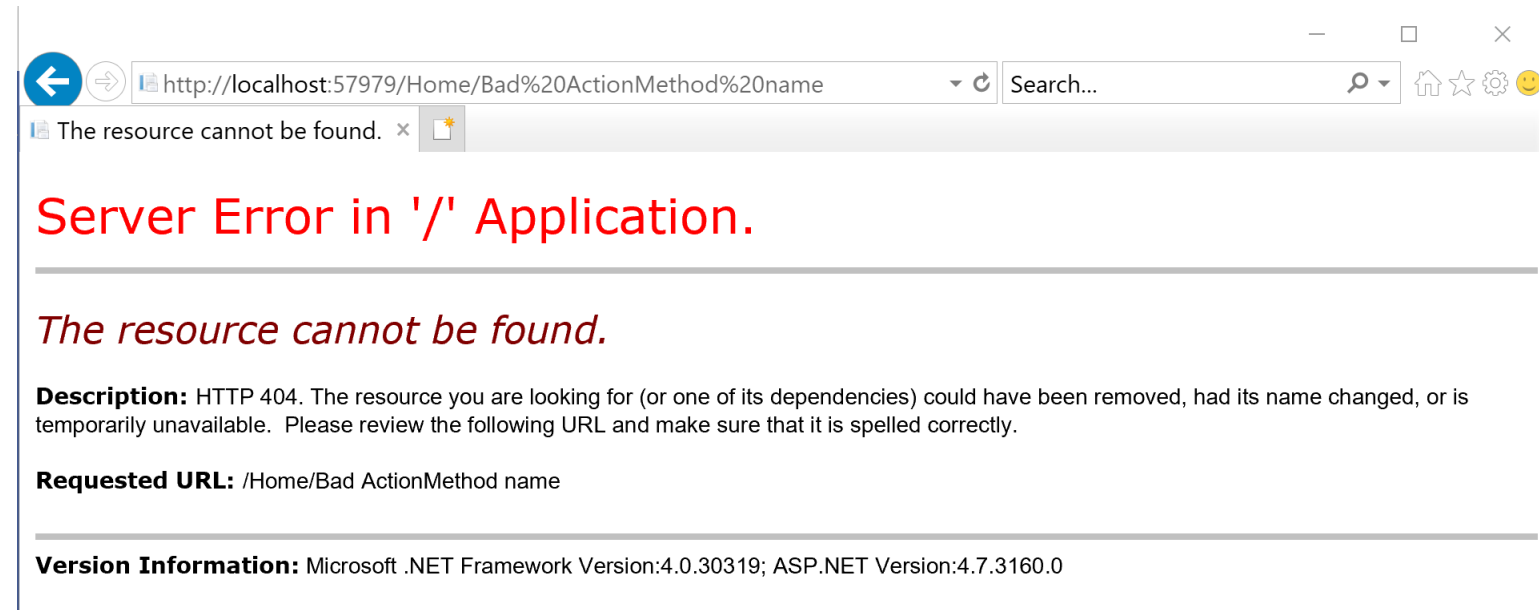
ELMAH IN ACTION

- Create an **ActionMethod**.

```
namespace DemoELMAH.Controllers
{
    0 references
    public class HomeController : Controller
    {
        0 references
        public ActionResult Index()
        {
            return View();
        }
    }
}
```

ELMAH IN ACTION

- Run the Project. Now, enter the wrong URL in address bar and hit Enter. We will get 404 Error.



ELMAH IN ACTION

- Now, for ELMAH Log, enter the below URL in address bar.

http://localhost:57979/elmah.axd

Error log for /LM/W3SVC/2/R...

Error Log for ROOT on ZALNET-PC

RSS FEED | RSS DIGEST | DOWNLOAD LOG | HELP | ABOUT

Errors 1 to 2 of total 2 (page 1 of 1). Start with [10](#), [15](#), [20](#), [25](#), [30](#), [50](#) or [100](#) errors per page.

Host	Code	Type	Error	User	Date	Time
ZALNET-PC	404	Http	A public action method 'Bad ActionMethod name' was not found on controller 'DemoELMAH.Controllers.HomeController'. Details...		15-10-2018	06:39
WS-NET-008	404	Http	The controller for path '/asd' was not found or does not implement IController. Details...		06-12-2016	07:55

Powered by [ELMAH](#), version 1.2.14706.955. Copyright (c) 2004, Atif Aziz. All rights reserved. Licensed under [Apache License, Version 2.0](#). Server date is Monday, 15 October 2018. Server time is 06:42:13. All dates and times displayed are in the Central European Summer Time zone. This log is provided by the XML File-Based Error Log.

ELMAH MAIL

- Setup Mail Server for getting every log in email; add the below code in **web.config**.

```
<!--START: ELMAH Setting -->
<elmah>

  <errorLog type="Elmah.XmlFileErrorLog, Elmah" logPath="~/ElmahLog" />
  <errorMail from="info@zalnet.pl"
    to="info@zalnet.pl"
    subject="Error - ELMAH demo"
    async="true" />
  <!--
    See http://code.google.com/p/elmah/wiki/SecuringErrorLogPages for
    more information on remote access and securing ELMAH.
  -->
  <security allowRemoteAccess="false" />
</elmah>
<!--END: ELMAH Setting -->

<!--START: ELMAH Email Setting -->
<system.net>
  <mailSettings>
    <smtp deliveryMethod="Network">
      <network host="host address" port="12345" userName="username here" password="password here"/>
    </smtp>
  </mailSettings>
</system.net>
<!--END: ELMAH Email Setting -->
```

ELMAH LOCATIONS

- **Store ELMAH logs on different locations.**
 1. Store ELMAH logs in XML file
- Create one folder named "ElmahLog" in root directory of your project. We use this folder for saving the XML file.

- Add the below setting in your **Web.config**.

```
<elmah><errorLog type="Elmah.XmlFileErrorLog, Elmah"
logPath="~/ElmahLog" /> </elmah>
```

2. Store ELMAH logs in RAM.

- Add the below setting in your **Web.config**.

```
<elmah><errorLog type="Elmah.MemoryErrorLog, Elmah"
size="100" /> </elmah>
```

.

ELMAH LOCATIONS

3. Store ELMAH logs in Microsoft SQL Server.

- Add the below setting in your Web.config.

```
<elmah><errorLog type="Elmah.SqlErrorLog, Elmah"
connectionString="DBEntities" /> </elmah>
```

- Don't forgot to add connectionString name as "DBEntities".

```
<connectionStrings><add name="DBEntities"
connectionString="data source=server name;initial
catalog=database name;persist security info=True;user id=your
username;password=your password;Trusted_Connection=True" />
</connectionStrings>
```

•

DEMO

Elmah

Q & A

Email: info@zalnet.pl

Demos after conference:

<https://github.com/bzalewa/ConferenceKrakow>

Thanks for coming 😊